Estrutura de dados Homogêneas

Slide baseado no tutorial do Professor Ailton Cruz, Algoritmo e Estrutura de Dados I

Estruturas Homogêneas Básicas

- Vetores (Utilizando Listas)
- Matrizes (Utilizando Listas)

Vetores

“Define-se um vetor através do seu **nome** e do **seu tamanho em termos da quantidade de elementos**. **Muitas vezes, o tamanho do vetor é atribuído antes de se conhecer a real quantidade de dados a serem armazenados e esse tamanho não muda até o término da execução do programa (alocação estática)**. **A modificação do tamanho do vetor durante a execução do programa é permitida quando a alocação é dinâmica**.” (CRUZ, 20xx)

Ex: nome_vetor [tamanho]

Slide baseado no tutorial do Professor Ailton Cruz, Algoritmo e Estrutura de Dados I

Vetor

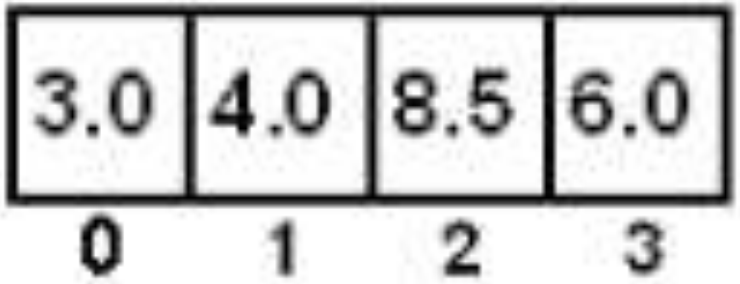
Iniciamos em … ?



Slide baseado no tutorial do Professor Ailton Cruz, Algoritmo e Estrutura de Dados I

Vetor

- Índice {0, 1, 2, 3 … , n-1)



Slide baseado no tutorial do Professor Ailton Cruz, Algoritmo e Estrutura de Dados I

Vetores (Pseudo)

- Atribuição de valores

nota[0] <- 3.0

nota[1] <- 4.0

nota[2] <- 8.5

nota[3] <- 6.0

Slide baseado no tutorial do Professor Ailton Cruz, Algoritmo e Estrutura de Dados I

Vetores (Pseudo)

- Atribuição de valores

```
nota <- [3.0, 4.0, 8.5, 6.0]
```

Slide baseado no tutorial do Professor Ailton Cruz, Algoritmo e Estrutura de Dados I

Vetores

- Leitura de valores

nota[0]

nota[1]

nota <- [3.0, 4.0, 8.5, 6.0]

Slide baseado no tutorial do Professor Ailton Cruz, Algoritmo e Estrutura de Dados I

Em python - Listas

```
vetor = [1, 2, 3]

for i in vetor:
    print(i)
```

Slide baseado no tutorial do Professor Ailton Cruz, Algoritmo e Estrutura de Dados I

Listas

- E se ...

```
vetor = [1, 2, "Teste"]
```

Lista

- minha lista **não** precisa conter somente os mesmos valores

```
vetor = [1, 2, "Teste"]

for i in vetor:
    print(i)
```

Slide baseado no tutorial do Professor Ailton Cruz, Algoritmo e Estrutura de Dados I

Listas

1. Inserindo itens na lista (append)

nome_lista.append(“valor”)

Slide baseado no tutorial do Professor Ailton Cruz, Algoritmo e Estrutura de Dados I

Listas

- Inserindo itens no final lista (append)

```
vetor = [1, 2, "Teste"]

for i in vetor:
  print(i)

vetor.append("Novo item")

for i in vetor:
  print(i)
```

Slide baseado no tutorial do Professor Ailton Cruz, Algoritmo e Estrutura de Dados I

Listas

- Inserindo itens em uma posição da lista (insert)

```
vetor = [1, 2, "Teste"]
vetor.insert(1, "Novo item")
for i in vetor:
  print(i)
```

Slide baseado no tutorial do Professor Ailton Cruz, Algoritmo e Estrutura de Dados I

Listas

- Removendo itens da lista (remove)

```
vetor = [1, 2, "Teste"]
vetor.remove("Teste")
for i in vetor:
  print(i)
```

Slide baseado no tutorial do Professor Ailton Cruz, Algoritmo e Estrutura de Dados I

Listas

- O que acontece?

```
vetor = [1, 2, "Teste", “Teste”]
vetor.remove("Teste")
for i in vetor:
  print(i)
```

Slide baseado no tutorial do Professor Ailton Cruz, Algoritmo e Estrutura de Dados I

Listas

- removendo da lista (pop)

```
vetor = [1, 2, "Teste"]
vetor.pop()
vetor.pop(0)
for i in vetor:
  print(i)
```

Slide baseado no tutorial do Professor Ailton Cruz, Algoritmo e Estrutura de Dados I

Listas

- Remove: Aceita o **valor do elemento** como argumento
- Pop: Remove com base no **index**

Slide baseado no tutorial do Professor Ailton Cruz, Algoritmo e Estrutura de Dados I

Outras (Filtrando por posições)

```
vetor = [1, 2, "Teste"]
print(vetor[:2])
```

Slide baseado no tutorial do Professor Ailton Cruz, Algoritmo e Estrutura de Dados I

Outras

- sort() -> Ordenar (**Mesmo tipo**) Ex: `lista = ["e", "a"] | lista = [1,2,3]`
- count(x) -> número de ocorrência
- reverse() -> inverte
- Etc …



Slide baseado no tutorial do Professor Ailton Cruz, Algoritmo e Estrutura de Dados I

Assim

Muito mais que um vetor.

- Recursos ricos e flexíveis

Slide baseado no tutorial do Professor Ailton Cruz, Algoritmo e Estrutura de Dados I

Também podemos manipular por posição

```
vetor = [4, 2, 5, 7]

vetor[3] = "Teste"

print(vetor)
```

Exibir apenas um “conjunto” de itens

```
vetor = [4, 2, 5, 7]

print(vetor[1: 4])
```

Se quiser que pegue de 1 até n-1:

```
vetor = [4, 2, 5, 7]

print(vetor[1:])
```

O inverso também é verdadeiro!

```
print(vetor[:2])
```

Split

```
nome = 'Hello World'

lista = nome.split(' ')

print(lista)
```

Split

E se quiser pegar o nome e sobrenome de joão?

```
nome = 'João Carlos da Silva'
```

Split

E se quiser pegar o nome e sobrenome de joão?

```
nome = 'João Carlos da Silva'

lista = nome.split(' ')

print(lista[0])
print(lista[3])


```

Aqui estou falando de um tamanho fixo! lista tem tamanho 4 … e se nome for input()?

```
nome = input('Digite seu nome completo: ')

```

Split

```
nome = input('Digite seu nome completo: ')

lista = nome.split(' ')

print(lista[0])
print(lista[-1])
```

Lembre-se!

- **Resolvemos problemas, por mais simples que pareçam; não devem ser desconsiderados.**

O modo como você reúne, administra e usa a informação determina se vencerá ou perderá. -Bill Gates

Efficient Arrays of Numeric Values

- https://docs.python.org/3/library/array.html

Vetores [Exercício em Sala]

1. Solicite que o usuário escreva 3 nomes e armazene em 1 lista utilizando **while.** Exiba os em ordem alfabética.
2. Leia uma lista de 10 posições. Contar e escrever quantos valores pares ele possui.
3. Crie um programa que lê 6 valores inteiros e, em seguida, mostre na tela os valores lidos na ordem inversa.
4. Dado a lista **números = [1,2,3,4, -1, 10, -4, 20]**, substitua onde for valor negativo por 0 -> Dica: Usar **for i in range(len(numeros))**

https://www.facom.ufu.br/~backes/gbt017/ListaPython04.pdf

Referências

1. https://educapes.capes.gov.br/bitstream/capes/176522/2/TextoAEDI_SI_UFAL_AiltonCruz.pdf

Slide baseado no tutorial do Professor Ailton Cruz, Algoritmo e Estrutura de Dados I